State

Season: 3 · Behavioral · Category: GoF · Level: Intermediate · Build with Waqar

Today's Hook

- Today: State as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



State structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for State
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/functional/state/State.java
package com.buildwithwaqar.patterns.functional.state;

import java.util.function.Function;

public record State<S, A>(Function<S, Result<S, A>> run) {
    public record Result<S, A>(S state, A value) {}

    public Result<S, A> run(S s) { return run.apply(s); }

    public <B> State<S, B> map(Function<? super A, ? extends B> f) {
        return new State<>(s -> {
            var r = run(s);
            return new Result<>(r.state(), f.apply(r.value()));
        });
    }

    public <B> State<S, B> flatMap(Function<? super A, State<S, B>> f) {
        return new State<>(s -> {
            var r = run(s);
            return f.apply(r.value()).run(r.state());
        });
    }
```

Implementation (2)

```
    public static <S, A> State<S, A> pure(A a) { return new State<>(s -> new Result<>(s, a)); }
    public static <S> State<S, S> get() { return new State<>(s -> new Result<>(s, s)); }
    public static <S> State<S, Void> set(S s) { return new State<>(ignored -> new Result<>(s, null)); }
    public static <S> State<S, Void> modify(Function<S, S> f) {
        return new State<>(s -> new Result<>(f.apply(s), null));
    }
}
```

Demo.main

```java
package com.buildwithwaqar.patterns.functional.state;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== State walkthrough ===");
    State<Integer, Integer> program = State.<Integer>get()
      .flatMap(n -> State.<Integer>set(n + 1).flatMap(x -> State.pure(n)));
    var r = program.run(10);
    System.out.println("value=" + r.value() + " state=" + r.state());
  }
}
```

Walkthrough Output

```
=== State walkthrough ===
start=NEW
afterPay=PAID
afterShip=SHIPPED
cancelOnShipped=IllegalStateException
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for State
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → State → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.